Transaction Isolation Levels



Isolation Levels

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

Data Phenomena

- Dirty Read
- Non-Repeatable Read
- Phantom Reads

Read Uncommitted IL

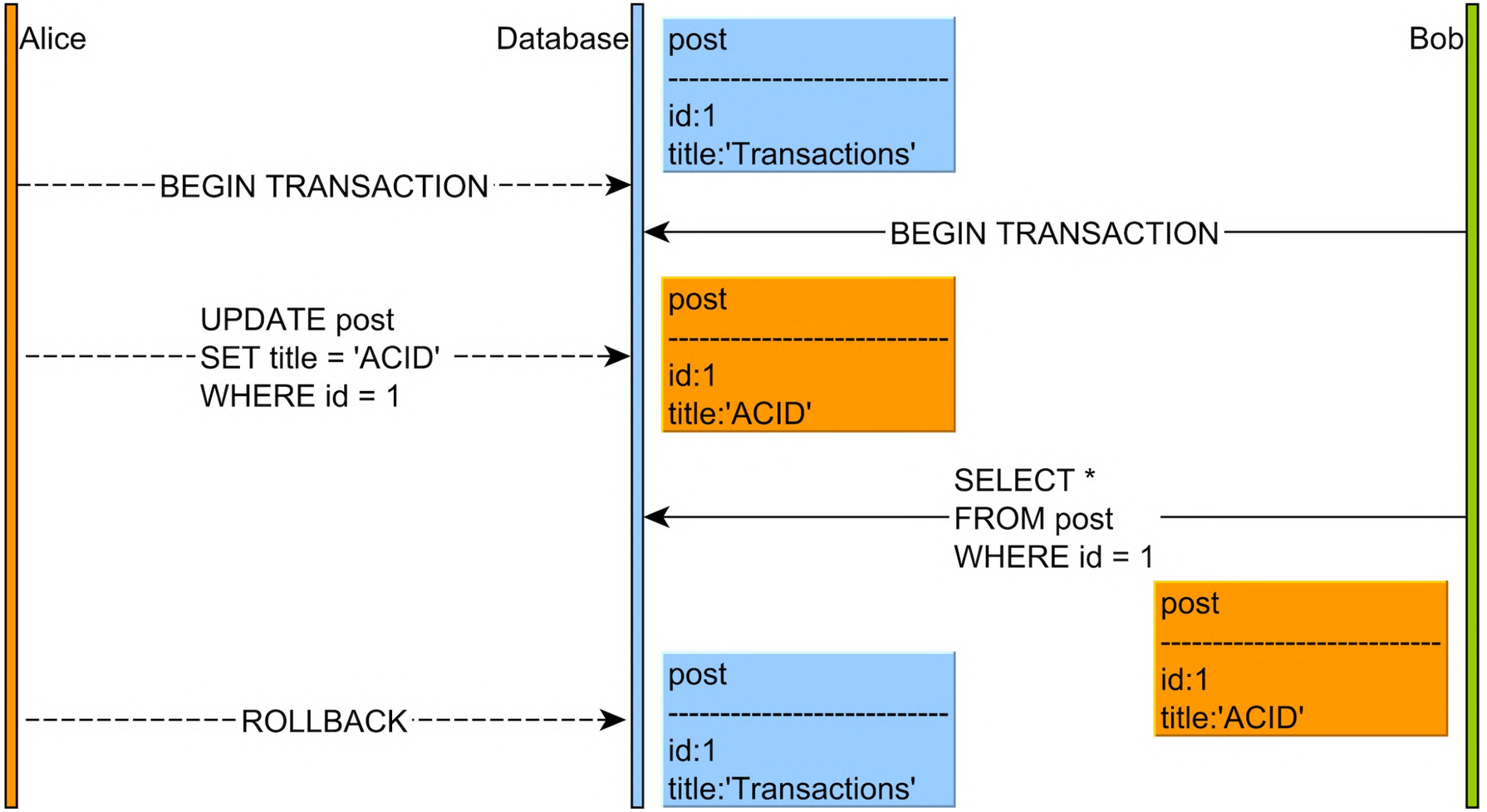
In the context of database isolation levels, Read Uncommitted is the lowest level of isolation. When a transaction is set to Read Uncommitted, it means that it allows reading data that has been modified but not yet committed by other concurrent transactions. This isolation level provides no protection against dirty reads, non-repeatable reads, and phantom reads, which are phenomena that can occur in concurrent environments.

Read Uncommitted should not be used in most cases.

Dirty Read

A "dirty read" is a term used in the context of database systems and transactions. It refers to a situation where one transaction reads data from a row in a database that has been modified by another transaction but has not been committed yet. This means that the data being read might be temporary and subject to change or rollback if the modifying transaction is later aborted.

Dirty Read


Alice
Database
Bob
post
---
id:1
title:'Transactions'
BEGIN TRANSACTION
BEGIN TRANSACTION
post
---
id:1
title:'ACID'
UPDATE post
SET title = 'ACID'
WHERE id = 1
SELECT *
FROM post
WHERE id = 1
post
---
id:1
title:'ACID'
post
---
id:1
title:'Transactions'
ROLLBACK

Read Committed IL

In the Read Committed isolation level, a transaction ensures that any data it reads or writes is not affected by other transactions until it commits. However, it allows other transactions to read the same data concurrently. It provides a higher level of isolation than Read Uncommitted, where transactions can read uncommitted changes made by other transactions (resulting in dirty reads).

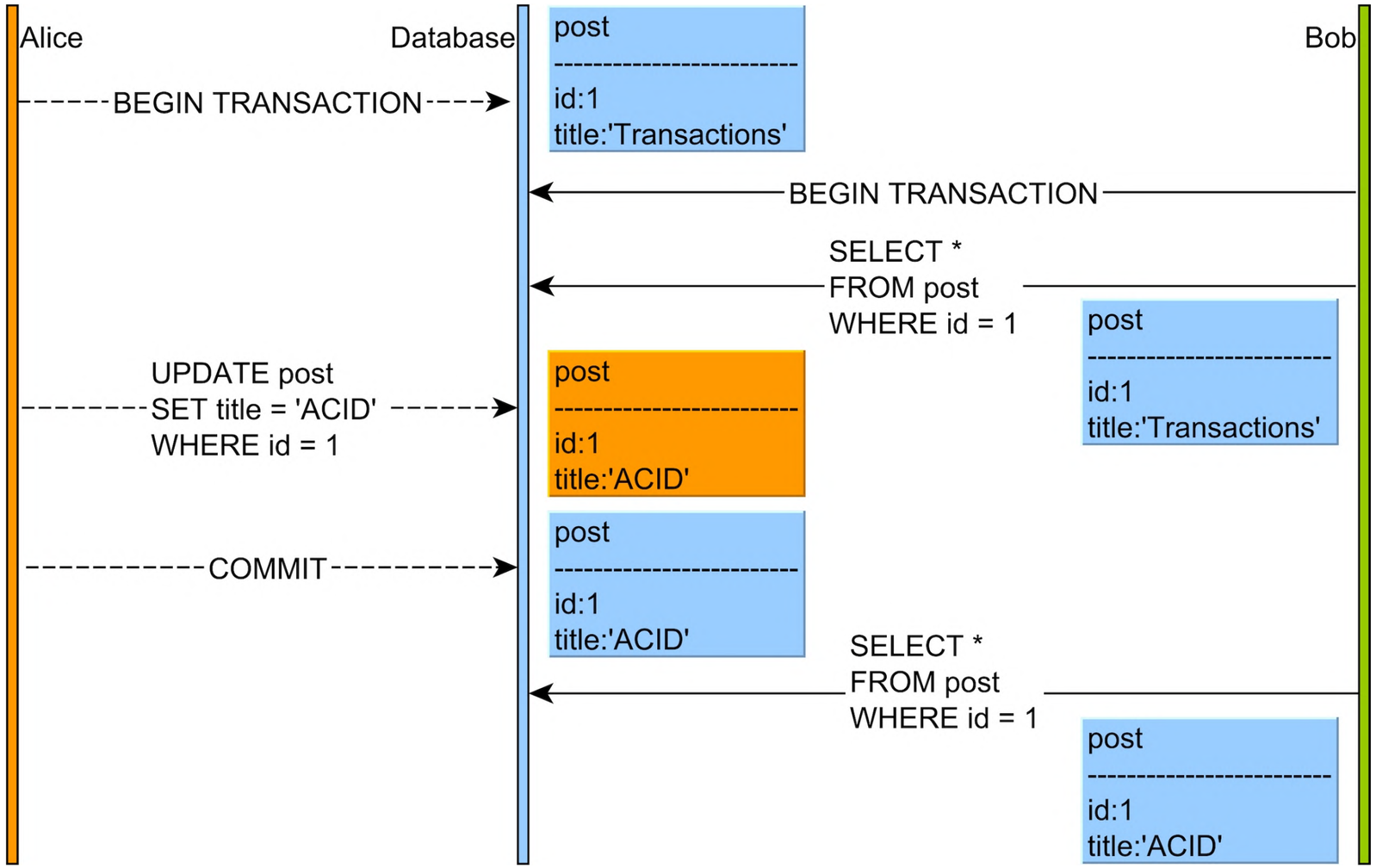
Read Committed is faster than stricter isolation levels such as Repeatable Read and Serializable, though it provides weaker consistency guarantees.

Non-Repeatable Read

Non-repeatable read is a term used in database systems to describe a phenomenon that can occur in certain isolation levels, particularly in systems that use concurrent transactions. Isolation levels determine the visibility and interaction between different transactions accessing the same data in a database.

In a non-repeatable read scenario, a transaction reads a row from a database and then, before the transaction is complete, another transaction modifies or deletes that same row. When the first transaction attempts to read the same row again, it gets a different value or finds that the row no longer exists. This inconsistency between the data read in two consecutive reads is what makes it "non-repeatable."

Non-Repeatable Read



Non-Repeatable Read vs Lost Update

1. Non-repeatable read occurs when a transaction reads a particular piece of data twice, but the value of the data changes between the two reads due to another committed transaction. This phenomenon violates the isolation property of the database transaction.
2. Lost update occurs when two or more transactions attempt to update the same data simultaneously, and one of the updates overwrites the changes made by the other transaction. This can lead to a loss of data integrity, as one transaction's update effectively "overwrites" the other transaction's changes, causing them to be lost.

Repeatable Read IL

In the context of Repeatable Read, a transaction ensures that if it reads a piece of data, it will see the same data if it reads it again later in the same transaction, regardless of any changes made by other transactions during that time.

When a transaction reads a piece of data, it acquires a read lock on the accessed data. This lock prevents other transactions from modifying the data until the reading transaction is complete. However, other transactions can still read the same data as long as they don't try to update it.

It's important to note that Rrepeatable Read can lead to increased concurrency issues, as transactions holding read locks for an extended period might block other transactions from writing or updating data. Additionally, it might also cause more deadlock scenarios.

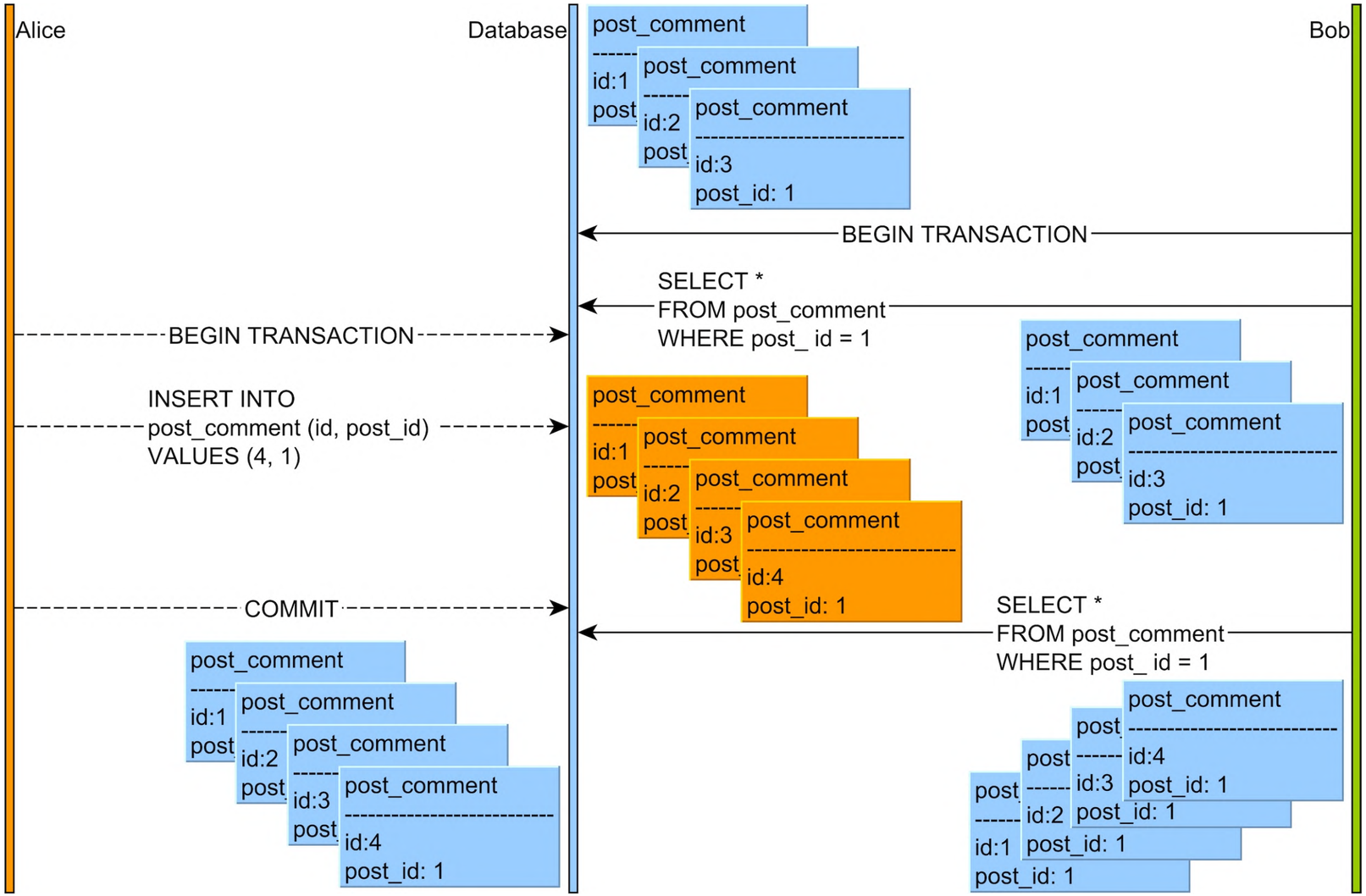
Phantom Read

Phantom read is a term used in the context of database transactions and concurrency control. It refers to a phenomenon where a transaction in a multi-user environment sees a set of data that appears to be consistent, but during the transaction's execution, another concurrent transaction modifies the data, leading to different results if the first transaction is re-executed.

In other words, a phantom read occurs when a transaction reads a range of rows that satisfy a certain condition, but while the transaction is still active, another transaction inserts, updates, or deletes rows that also meet the same condition, causing the initial transaction to see additional "phantom" rows when it re-executes the same query.

DRIM

Phantom Read



Serializable IL

The Serializable isolation level ensures that transactions appear to be executed in serial order, one after another, even if they are executed concurrently. This means that the result of executing transactions concurrently is guaranteed to be the same as if they were executed sequentially, without any overlapping.

To achieve serializable isolation, DBMS employs locking and other concurrency control mechanisms to prevent conflicts between transactions. These mechanisms ensure that a transaction doesn't read or write data that is being modified by another transaction concurrently. As a result, the database guarantees that transactions are isolated from each other, and their effects are indistinguishable from running sequentially.

However, it's essential to note that achieving Serializable isolation can come with performance trade-offs. The locking and synchronization mechanisms employed to maintain serializability may introduce contention and reduce concurrency, potentially impacting the system's overall throughput.

Serializability and Linearizability

Serializability and Linearizability are two different consistency models used in distributed systems to define how concurrent operations are executed and how the system ensures correctness and consistency of data.

- **Serializability** is a consistency model that ensures that the outcome of concurrent transactions in a distributed system is equivalent to some serial execution of those transactions. In other words, it guarantees that the system behaves as if transactions were executed sequentially, one after another, even though they might have been executed concurrently. This ensures that the final state of the system is consistent and complies with the intended constraints.

- **Linearizability**, also known as atomic consistency, is a stronger consistency model than serializability. It guarantees that each operation in a distributed system appears to have occurred instantaneously at a single point in time between its invocation and completion. In simpler terms, linearizability ensures that concurrent operations are equivalent to some sequential execution where the operations seem to happen instantly one after another.

Isolation Levels and Data Phenomena

	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	allowed	allowed	allowed
Read Committed	prevented	allowed	allowed
Repeatable Read	prevented	prevented	allowed
Serializable	prevented	prevented	prevented

Default Isolation Levels

Database	Default Isolation Level
Oracle	Read Committed
MySQL	Repeatable Read
MS SQL Server	Read Committed
PostgreSQL	Read Committed
DB2	Cursor Stability

Cursor Stability

Cursor stability is a consistency model which strengthens *read committed* by preventing *lost updates*. It introduces the concept of a *cursor*, which refers to a particular object being accessed by a transaction. Transactions may have multiple cursors. When a transaction reads an object using a cursor, that object cannot be modified by any other transaction until the cursor is released, or the transaction commits.

MVCC IL Implementation

MVCC implements isolation levels using an optimistic concurrency approach. It means that MVCC algorithms do not block concurrent transactions. Instead, they detect inconsistencies at the commit phase and roll back a transaction that broke consistency guarantees.

It is vitally important to understand whether an pessimistic or optimistic concurrency is used. It will lead to different compensation code in the application.

DB-specific Implementations



Keep in mind that the specific behavior and support for isolation levels may vary between different database management systems. Therefore, it is essential to refer to the documentation of the specific database system you are using to understand how they implement and handle different isolation levels.

Transactional Outbox

The transactional outbox pattern is a design pattern used in distributed systems to ensure reliable, scalable, and consistent message-based communication between services. It addresses the challenges that arise when multiple services need to be notified or updated as a result of a single action (e.g., creating an order, updating user information) while maintaining data consistency and avoiding data loss or duplication.

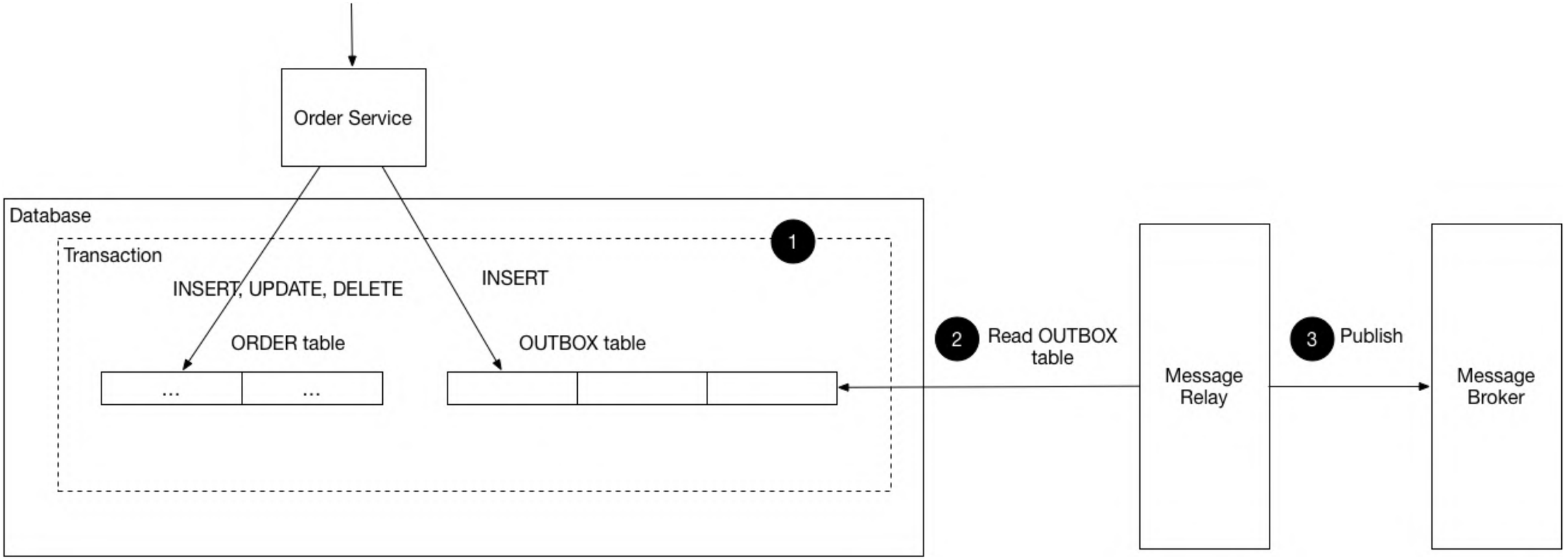
Transactional Outbox

Here's how the transactional outbox pattern works:

- **Outbox Table**: Each service that needs to communicate with others maintains an "outbox" table within its own database. This table stores the messages that need to be sent to other services. Each row in the table represents a single message, including the payload and metadata needed for delivery.
- **Atomic Transaction**: When the primary action (e.g., creating an order) occurs within a service, the database transaction becomes "atomic" by combining the primary action and the insertion of a new message into the outbox table. This ensures that either both the primary action and the message insertion succeed, or both fail.
- **Background Processing**: After the primary action is successfully committed to the database, a background process or message queue consumer within the same service takes responsibility for reading the messages from the outbox table and dispatching them to the appropriate destination services. This step ensures that the sending of messages doesn't affect the response time of the main operation.
- **Message Delivery**: The background process delivers the messages to the intended recipients (other services) through message queues or other communication channels. The receiving services process these messages independently and act accordingly, updating their own state or performing other relevant actions.

DRIM

Transactional Outbox



DRIM

Thank you! Questions?

